

TSAD-Orchestra: An Agentic Approach to Time-Series Anomaly Detection

Georgios Atmatzidis
Aristotle University of Thessaloniki
AEM: 234
gatmatza@csd.auth.gr

Dimitrios Litsidis
Aristotle University of Thessaloniki
AEM: 216
dlitsida@csd.auth.gr

ABSTRACT

Time series are sequences of observations collected over time and are central to modern applications such as industrial monitoring, healthcare, finance, and cybersecurity systems. In these settings, identifying abnormal temporal behavior is critical, since anomalies often indicate faults, attacks, rare events, or early signs of system degradation. Time-Series Anomaly Detection (TSAD) aims to automatically detect such unusual patterns while accounting for temporal dependencies, non-stationarity, seasonality, and noise.

In this project, we propose **TSAD-Orchestra** [2], a novel agentic model ensemble framework designed to dynamically navigate the complexities of Time-Series Anomaly Detection. Our system implements a **Model Context Protocol (MCP) server** equipped with custom-built tools that empower an AI agent to perform informed orchestration. By profiling time-series characteristics, the agent dynamically screens available detectors, filters out uninformative models, and performs precision drill-downs on suspicious temporal regions. The framework seamlessly integrates model selection with ensembling: the agent fuses the highest-quality signals into a robust, final anomaly score. To ensure reliability and prevent hallucinations, we introduce a **dual-agent architecture**: a Primary Agent manages the exploratory analysis and selection process, while a secondary **Validator Agent** rigorously scrutinizes the primary agent's logic against deterministic tool outputs, forcing iterative self-correction when necessary.

KEYWORDS

AutoML, Agentic AI, Time-Series, Time-Series Anomaly Detection

1 INTRODUCTION

Advances in sensing, networking, storage, and processing technologies have enabled data acquisition at massive scale, with particular emphasis on time series [5]. In the literature, several time-series mining objectives have been extensively studied, including clustering [10], similarity search [13], anomaly detection [5], and classification [9].

Time-series Anomaly Detection (TSAD) focuses on identifying observations or subsequences that deviate significantly from expected temporal behavior, enabling early detection of critical events such as equipment failures, fraud, cyberattacks, and medical abnormalities.

Motivating Example. While data scientists have created a diverse toolkit of anomaly detection algorithms, each tailored to handle specific patterns, finding the right fit remains a significant challenge. The "perfect" solution exists in theory through precise parameter tuning and high-quality training data, but the reality is often more chaotic. Between the sheer volume of available models

and a frequent shortage of domain expertise or labeled data, successful detection often comes down to manual trial and error and a fair amount of luck.

Finding a "one-size-fits-all" solution for time-series anomaly detection remains a major challenge because detection accuracy, measured by metrics such as *VUS-PR*, varies substantially across domains. As illustrated in Figure 1, derived from [6], the optimal algorithm depends on the data domain. For instance, **MOMENT** performs best in the *Facility* domain, **LSTMAD** is superior in *HumanActivity*, and **KShapeAD** leads in *Medical* applications. These inconsistencies show that relying on manual trial and error or a single default algorithm is insufficient. Consequently, there is a critical need for a context-aware model selection framework that can dynamically navigate these trade-offs. Such a framework should perform informed orchestration by analyzing time-series features and the requirements of the target domain.

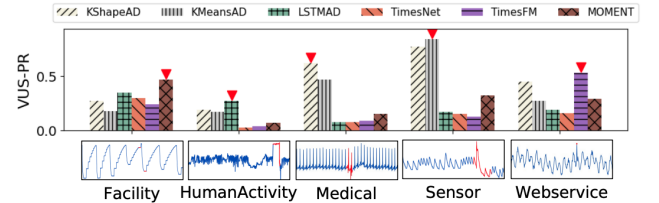


Figure 1: Detection accuracy (VUS-PR) for six representative anomaly detectors across five domains in the TSB-AD benchmark. The red triangle indicates the model with the best detection accuracy.

2 RELATED WORK

Agentic TSAD. Recent advances in time-series anomaly detection (TSAD) have shifted toward agentic frameworks that use reasoning to solve complex tasks. A key example is *AnomaMind* [12], which employs an AI agent to select and apply various tools for identifying anomalies. While our work follows a similar agentic direction, it differs in implementation. *AnomaMind* relies on specialized reinforcement learning (RL) and model training to optimize detection decisions, which requires significant computational resources. In contrast, our framework focuses on tool-based orchestration and selection among off-the-shelf TSAD methods.

Labeled Data. In anomaly detection, defining ground truth is inherently difficult, since behavior considered anomalous in one context may be normal in another. This makes rigorous evaluation challenging. To address this issue, we rely on benchmark collections with manually annotated anomalies. *TimeEval* [14] provides

an evaluation framework together with labeled time-series datasets, while *TSB-UAD* [7] offers a large and diverse set of labeled time series from multiple application domains. Because these datasets are publicly available, there is a risk of data leakage and overly optimistic performance estimates. To mitigate this risk, we anonymize dataset identifiers where applicable and apply additional safeguards in our evaluation protocol.

Inspiration from DS-STAR. TSAD-Orchestra is inspired by the agentic architecture of DS-STAR [8]. While DS-STAR generalizes across the data science life cycle, our framework specializes this approach for the constraints of time-series anomaly detection. We adopt a similar multi-agent philosophy through a dual-agent architecture: a primary agent manages context-aware selection, while a secondary validator agent scrutinizes the logic and reasoning of the first. This mirrors the *Verifier* loop in DS-STAR and improves robustness. Furthermore, TSAD-Orchestra can act as an iterative *Debugger*, dynamically pivoting between candidate algorithms or constructing composite ensembles to satisfy operational requirements.

3 PROPOSED METHODOLOGY

Data Persistence

Our initial experiments draw from the TSB-UAD benchmark [7], using a randomly sampled batch of univariate time series chosen to cover all available domains and a representative range of lengths. To support efficient profiling, we store the time series in a Dockerized TimescaleDB instance, allowing the feature-extraction tools (seasonality, time-series length etc.) exposed through the MCP server to be implemented as SQL queries and executed with minimal latency. Reproducibility is ensured through setup scripts that automate container initialization and database population with a fixed dataset.

Anomaly Detectors

Following the ideas of [11], we consider, as a start, 5 anomaly detectors of varying complexity: Local Outlier Factor (LOF), Isolation Forest, Polynomial Fit, Principal Component Analysis (PCA), and Histogram-based Outlier Score (HBOS). While LOF and Isolation Forest are well-established anomaly detection methods, Polynomial Fit, PCA, and HBOS need a brief explanation. Polynomial Fit models the underlying global trend of the time series using a low-degree polynomial, scoring anomalies based on their residual deviation from this fitted curve. PCA projects data onto a lower-dimensional hyperplane and scores anomalies by their reconstruction distance from that plane. HBOS takes a simpler, histogram-based approach: it partitions the feature space into bins and assigns each point an anomaly score inversely proportional to the density of its bin, under the assumption that anomalies fall in sparsely populated regions.

Each anomaly detector is exposed as a tool in the MCP server, with detailed docstrings that guide the agent in selecting the most appropriate algorithm for a given context. For instance, if the agent determines that the input time series contains fewer than 20 points via its profiling tools, it will automatically exclude LOF, which requires a minimum of 20 observations to operate correctly.

Performance Optimizations: MCP Score Caching

A significant bottleneck in automated anomaly detection workflows is the repetitive execution of expensive machine learning models, especially when the agent iteratively drills down into different segments of the same dataset or when the validator agent requests re-evaluations. To address this, we implemented an intelligent caching mechanism within our Model Context Protocol (MCP) server. When a detector tool is first invoked for a specific time series, the backend computes the anomaly scores across the entire dataset and caches these raw scores locally. Subsequent tool calls—whether they are targeted drill-down queries for specific time windows, validator verifications, or final ensemble fusions—retrieve the precomputed scores from the cache rather than re-running the underlying algorithm. This optimization reduces computational overhead, accelerates the agent’s iterative reasoning loop, and ensures that ensemble scoring (which fuses the cached scores of the selected informative detectors) operates with lower latency.

LLM Harness: LangGraph

TSAD-Orchestra uses LangGraph [1] as the execution layer that coordinates the agent’s reasoning loop and tool usage. Rather than relying on a single monolithic prompt-response interaction, we model the system as a state graph in which the conversation history is carried through each step of execution. The graph maintains a structured state containing the accumulated messages and the final anomaly detection report, which allows the agent to reason incrementally and preserve context across tool calls. The resulting state graph is illustrated in Figure 2.

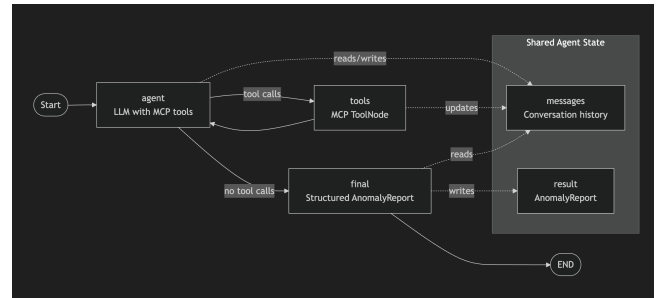


Figure 2: The LangGraph state graph used for agent orchestration in TSAD-Orchestra. The graph maintains a structured state containing the accumulated messages and the final anomaly detection report.

Initially, the workflow follows a simple yet effective loop: the *agent* node queries the language model, the graph inspects the last message to determine whether a tool was requested, and, if so, control is routed to a *tools* node implemented with LangGraph’s ToolNode. This node executes the MCP tools exposed by our server and returns the results to the agent, enabling iterative reasoning over the time-series profile and detector selection. Once the model no longer requests tools, the graph transitions to a final node that converts the full message history into a structured *AnomalyReport* using schema-constrained output generation.

We chose LangGraph because it separates reasoning, tool execution, and final report generation into explicit nodes, which improves debuggability and allows the workflow to be extended easily. In our implementation, this is particularly useful because the agent must combine language-model reasoning with external tools exposed through the MCP server while still producing a validated final result. As a result, LangGraph provides a clear control flow for agentic time-series anomaly detection and a practical foundation for future extensions such as additional validation steps.

LangSmith for Observability and Debugging

LangSmith [1] serves as the observability layer for our LangGraph-based agent, making each run transparent and auditable through structured execution traces. Setup is minimal: a few environment variables enable automatic logging of every agent run without touching application logic, which is valuable in an agentic workflow.

For each execution, LangSmith captures end-to-end workflow traces, hierarchical run trees that mirror the LangGraph node structure, per-step latency breakdowns, tool invocation order, and token usage statistics.

These traces serve us two purposes. Operationally, they allow us to identify bottlenecks, compare prompt and tool strategies, and diagnose failure modes such as redundant tool calls or premature loop termination. Scientifically, they support reproducibility by tying every reported result back to the exact sequence of model calls and tool invocations that produced it, turning TSAD-Orchestra from a black box into an accountable, observable system.

System Prompts

The behavior of TSAD-Orchestra is shaped by a set of system and user prompts for both the Primary and Validator agents. The Primary Agent’s system prompt specifies its four-phase operation: Screening, Drill-Down, and Store Ensemble, guiding it to profile the data, identify informative detectors, and extract verified anomalies. The Validator Agent’s prompt provides strict criteria for accepting or rejecting the final report based on tool usage and evidence. The complete prompts can be found in Appendix A.

User Interface and Human-Agent Interaction

To facilitate demonstration and interactive evaluation, we developed a Streamlit-based interface for TSAD-Orchestra, which is publicly accessible at the time of writing [3]. The left panel exposes a dropdown menu for selecting any time series from our database, alongside descriptive statistics such as data points, mean, standard deviation, and range. The main panel provides an interactive Plotly visualization of the selected series and a control button to trigger the anomaly detection pipeline. Once detection completes, identified anomalies are overlaid directly on the plot. A results section details the number of anomalies found, detectors used, and tools called, offering expanders to view the agent’s textual summary, exact anomaly points, and a complete trace of all tool invocations.

Crucially, the interface is explicitly designed to bridge the gap between complex Automated Machine Learning (AutoML) pipelines and users lacking a formal data science background. Rather than overwhelming the user with hyperparameter tuning or algorithmic theory, the agent acts as an analytical translator. It generates a

natural language summary that explains in plain text why certain algorithms were chosen, what the detected anomalies represent in the context of the data profile, and the consensus behind the findings. To further enhance this communication, the system paves the way for conversational interaction, allowing users to dynamically request tailored visualizations, such as plotting specific time ranges, highlighting the raw anomaly scores of individual detectors, or visually contrasting normal seasonal patterns against anomalous spikes. This interactive, explainable visual dialogue shifts the focus from mere model selection to empowering non-experts to intuitively understand and trust their data.

Agentic Time Series Anomaly Detection Algorithm

Algorithm 1 LLM-Orchestrated Time Series Anomaly Detection

Require: Time series X , Set of detectors $\mathcal{D}$, Primary Agent $\mathcal{A}_p$, Validator Agent $\mathcal{A}_v$

Ensure: Final Anomaly Report $\mathcal{R}$

Phase 1: Profiling & Screening

- 1: $\mathcal{P} \leftarrow \text{profile_time_series}(X)$
- 2: $\mathcal{D}_{\text{run}} \leftarrow \mathcal{A}_p.\text{SelectDetectors}(\mathcal{P}, \mathcal{D})$ ▷ Run 2–3 detectors based on profile
- 3: **for** each detector $d \in \mathcal{D}_{\text{run}}$ **do**
- 4: $\text{stats}_d \leftarrow \text{RunDetector}(d, X)$ ▷ Returns hot segments & score percentiles
- 5: **end for**
- 6: $\mathcal{D}_{\text{informative}} \leftarrow \mathcal{A}_p.\text{AssessQuality}(\{\text{stats}_d\})$ ▷ Exclude flat/uninformative signals

Phase 2: Precision Drill-down

- 7: $\mathcal{H}_{\text{suspect}} \leftarrow \mathcal{A}_p.\text{IdentifySuspiciousSegments}(\{\text{stats}_d\})$
- 8: $\mathcal{C}_{\text{all}} \leftarrow \emptyset$
- 9: **for** each segment $[start, end] \in \mathcal{H}_{\text{suspect}}$ **do**
- 10: $\mathcal{C}_{\text{seg}} \leftarrow \text{drill_down_range}([start, end], \mathcal{D}_{\text{informative}})$
- 11: $\mathcal{C}_{\text{all}} \leftarrow \mathcal{C}_{\text{all}} \cup \mathcal{C}_{\text{seg}}$

Phase 3: Ensemble Storage

- 12: **end for**
- 13: $\mathcal{E} \leftarrow \text{store_ensemble_scores}(\mathcal{D}_{\text{informative}})$ ▷ Fusion using only informative detectors

Phase 4: Report Generation

- 14: $\text{summary} \leftarrow \mathcal{A}_p.\text{GenerateFinalReport}()$
- 15: $\mathcal{R}.\text{anomalies} \leftarrow \text{ExtractFromToolHistory}(\mathcal{C}_{\text{all}})$ ▷ Deterministic parsing
- 16: $\mathcal{R}.\text{summary} \leftarrow \text{summary}$
- 17: $\mathcal{R}.\text{detectors} \leftarrow \mathcal{D}_{\text{informative}}$
- 18: $\mathcal{R}.\text{scores} \leftarrow \mathcal{E}$

Phase 5: Validator Feedback Loop

- 19: $\text{isValid, critique} \leftarrow \mathcal{A}_v.\text{Validate}(\mathcal{R}, \{\text{stats}_d\}, \mathcal{C}_{\text{all}})$
 - 20: **if not** isValid **then**
 - 21: $\mathcal{A}_p.\text{AppendContext}(\text{critique})$
 - 22: **goto** Phase 1 ▷ Iterate agent reasoning until accepted
 - 23: **end if**
 - 24: **return** $\mathcal{R}$
-

The core of TSAD-Orchestra is a LLM-orchestrated workflow designed to automate the selection, execution, and synthesis of various time series anomaly detection algorithms. To achieve both high accuracy and explainability, the framework utilizes a dual-agent architecture comprising a Primary Agent ($\mathcal{A}_p$) for exploratory analysis and a Validator Agent ($\mathcal{A}_v$) for quality assurance. The detection process evaluates an input time series X using a dynamic subset of available detectors $\mathcal{D}$ and is structured into five distinct phases, as detailed in Algorithm 1.

Phase 1: Profiling & Screening

The algorithm begins by generating a statistical profile $\mathcal{P}$ of the input time series X . The Primary Agent ($\mathcal{A}_p$) evaluates this profile to dynamically select 2-3 candidate algorithms. For each selected detector, the system computes anomaly scores and returns a compact statistical summary containing score percentiles and "hot segments" (regions exhibiting the highest anomaly scores). The agent assesses the signal quality, distinguishing between informative detectors (those with sharp anomaly spikes) and uninformative ones (those with flat distributions), dropping the latter to avoid noise.

Phase 2: Precision Drill-down

Relying on the summarized hot segments, the Primary Agent identifies the most suspicious time regions ($\mathcal{H}_{\text{suspect}}$) requiring deeper inspection. For each identified segment, the system performs a localized drill-down query across the informative detectors via the `drill_down_range` tool. By analyzing these narrow windows, the system pinpoints consensus anomaly points (C_{seg}), indices flagged by multiple detectors simultaneously.

Phase 3: Ensemble Storage

Based on the behavioral insights gathered, the Primary Agent evaluates the performance and agreement of the informative detectors. It strategically calls the `store_ensemble_scores` tool, passing only the selected high-quality detectors. Crucially, the final anomaly score is not derived from a simple arithmetic average. Instead, the framework places significant emphasis on dynamic weighting for each selected detector based on its signal quality. Detectors that exhibit sharp, highly concentrated anomaly spikes (high signal concentration) are automatically assigned a heavier weight, while those providing flatter, less confident score distributions are significantly discounted. This dynamic fusion process produces a quality-weighted ensemble score vector E that aims to suppress noise and highlight the true anomaly signal, which is then saved to disk for subsequent rendering.

Phase 4: Report Generation

To assemble the final anomaly report $\mathcal{R}$, the workflow combines probabilistic LLM reasoning with deterministic parsing. The Primary Agent generates a natural language summary explaining its findings, the nature of the anomalies, and the rationale behind its detector filtering. Simultaneously, the exact timestamps of the anomalies are deterministically parsed and extracted from the structured tool history (C_{all}), bypassing the LLM to prevent hallucinations.

Phase 5: Validator Feedback Loop

To prevent hallucinations, the framework incorporates an autonomous self-reflection mechanism managed by the Validator Agent ($\mathcal{A}_v$). The Validator cross-references the Primary Agent's report against the underlying detector statistics, ensuring that all reported anomalies appear in the drill-down tool results, that an ensemble was properly stored, and that at least two detectors were initially screened. If the findings are deemed invalid or lack sufficient evidence, the Validator issues a targeted critique. This feedback is appended to the Primary Agent's context, triggering a loop back to Phase 1. This iterative cycle continues until the Validator approves the analysis.

End-to-End Example: Detecting Server CPU Spikes

To illustrate the complete workflow of our framework, consider a scenario where the system is tasked with analyzing a time-series dataset named `cpu_util_01`. This dataset contains 5,000 data points tracking minute-by-minute CPU utilization.

Step 1: Initialization. The Primary Agent is initialized to analyze `cpu_util_01`.

Step 2: Profiling & Screening (Phase 1). The agent uses `profile_time_series` and determines Isolation Forest and Local Outlier Factor are appropriate. It runs them and receives compact stat blocks indicating "hot segments" (e.g., around index 3200). It assesses both as informative.

Step 3: Precision Drill-Down (Phase 2). Observing the hot segment, the agent investigates the region between indices 3200 and 3299 by calling the `drill_down_range` tool. The backend isolates this subset and checks for agreement, returning consensus points at index 3245, 3246, and 3247.

Step 4: Ensemble Storage (Phase 3). The agent calls `store_ensemble_scores` with the informative detectors. The backend fuses the cached scores across all 5,000 points and saves this unified score vector to disk.

Step 5: Report Generation (Phase 4). The system deterministically extracts the consensus anomalies from the tool history. The agent writes a natural language summary explaining the identified bottleneck.

Step 6: Validator Feedback (Phase 5). The drafted report is routed to the Validator Agent. The Validator automatically reviews the trace, confirming that drill-downs were performed and anomalies match the tool outputs. The Validator approves the report, finalizing the execution.

4 EXPERIMENTS

To evaluate the effectiveness of TSAD-Orchestra, we conducted preliminary experiments using diverse datasets from the TSB-UAD benchmark. Specifically, we evaluated the framework on a curated subset of 45 time series originating from 7 distinct domains, with each series containing an average of approximately 3,000 data points. The objective was to assess whether the agentic framework could correctly profile time series, selectively apply appropriate detectors, and accurately identify anomalies while rejecting false positives, while measuring its overhead in terms of execution time and token usage.

4.1 Detection Performance

Figure 3 illustrates the average VUS-ROC across various detection methods. We selected the Volume Under the Surface (VUS-ROC) as our primary evaluation metric, as recent literature establishes that such threshold-independent measures provide a more robust and objective standard for evaluating range-based anomalies in time series compared to traditional point-based metrics [4]. As shown, individual base detectors (such as HBOS, IForest, LOF, PCA, and POLY) exhibit lower average performance due to their lack of adaptability across different domains. A simple ensemble of these detectors marginally improves robustness but remains vulnerable to noise from uninformative models. In contrast, the Agentic approach (TSAD-Orchestra) shows an improvement in VUS-ROC. By dynamically screening out uninformative detectors and strategically weighting the remaining signals based on dataset profiles, the agent produces more refined anomaly scores.

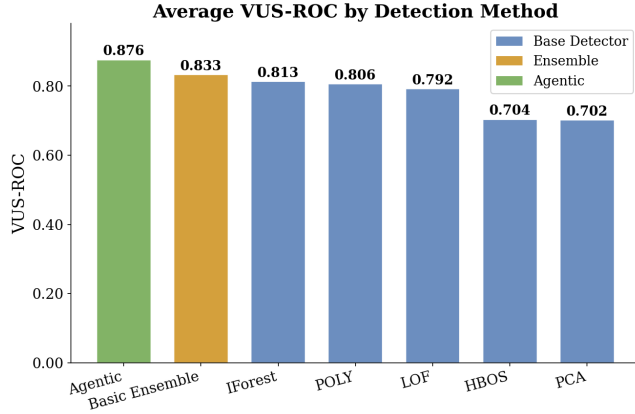


Figure 3: Average VUS-ROC by Detection Method. The agentic approach shows competitive performance compared to individual detectors and a basic ensemble.

4.2 Execution Time Overhead

While the agentic framework improves accuracy, it introduces computational overhead. Figure 4 presents the average execution time per dataset. Base detectors evaluate series in fractions of a second. The Agentic approach requires more time per dataset due to multiple LLM reasoning cycles and backend tool invocations. The inclusion of the Validator Agent adds a slight temporal overhead compared to the system without a validator (Agentic w/o valid.), representing a necessary trade-off for ensuring report accuracy and intercepting hallucinations.

4.3 Token Consumption and Validator Ablation

LLM token usage is a critical metric for operational cost. Figure 5 compares the average token usage per dataset between the Agentic method with and without the Validator Agent. Operating the full dual-agent architecture increases token consumption because the Validator consumes tokens to cross-reference tool outputs and occasionally forces the Primary Agent to iterate. Despite this increase,

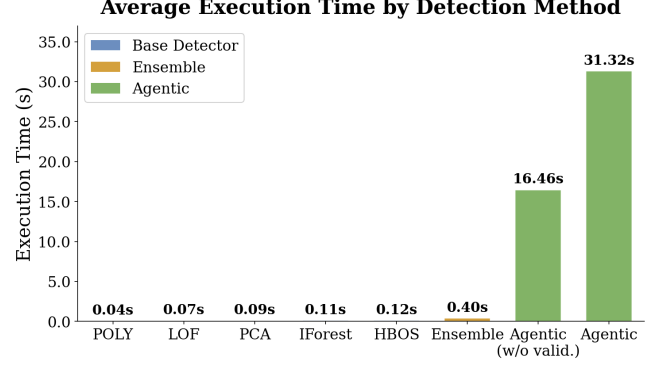


Figure 4: Average Execution Time by Detection Method. The LLM orchestration introduces additional execution time.

the summarization of time series into compact statistical blocks successfully prevents context window overflow.

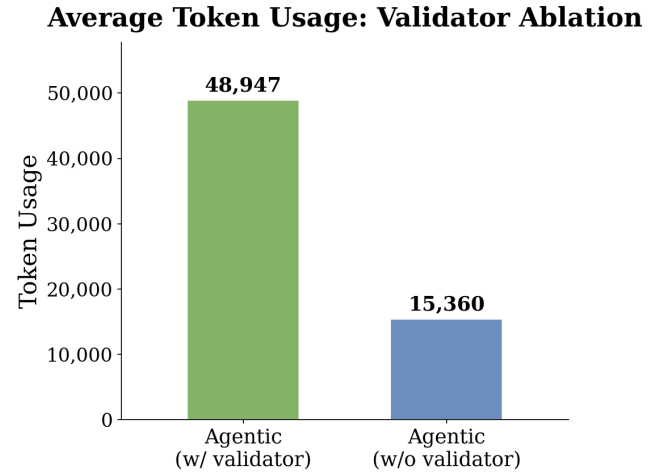


Figure 5: Average Token Usage comparing the Agentic framework with and without the Validator.

4.4 Tool Usage Analysis

Finally, we analyzed how frequently the agent invokes its available tools (Figure 6). In both configurations, the agent reliably uses the `profile_time_series` tool initially, followed by calls to screening detectors and the `drill_down_range` tool. When the Validator is active, we observe an increase in the average number of tool calls per dataset. This reflects instances where the Validator rejected an unsupported anomaly report, prompting the Primary Agent to re-evaluate the data and perform additional tool calls to satisfy the strict evidence criteria.

In summary, our trials confirm that TSAD-Orchestra successfully recognizes distinct data characteristics, excludes inappropriate detectors, and synthesizes accurate results. Filtering out uninformative detectors prior to ensemble scoring produces cleaner final

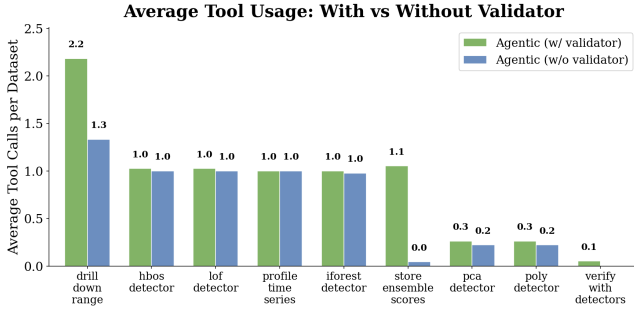


Figure 6: Average Tool Calls per Dataset. The active Validator occasionally triggers additional tool usage through feedback loops.

scores, and the Validator enforces reasoning rigor at the cost of increases in time and tokens.

5 DISCUSSION AND CONCLUSIONS

In this project, we introduced TSAD-Orchestra, an agentic model selection framework for time-series anomaly detection. By leveraging a dual-agent architecture and an MCP server, the system dynamically navigates detector selection through a structured pipeline of Screening, Drill-Down, and Ensemble Storage, while minimizing hallucinations through validation.

Despite these promising architectural steps, our overarching verdict is that agentic AI is not yet a mature, standalone solution for Time-Series Anomaly Detection. The considerable execution overhead, context window limitations, and the fragility of LLM reasoning when interpreting raw numerical arrays underscore why there is currently very little literature at the intersection of Agentic AI and TSAD. While our framework successfully automates parts of the model selection process and improves explainability, the heavy reliance on external deterministic tools proves that LLMs cannot yet natively "understand" time-series nuances without extensive, handcrafted statistical translation.

6 FUTURE WORK

To bridge the gap between current limitations and fully autonomous TSAD systems, several avenues of future research emerge:

- **Compact Time-Series Representations:** A more compact, task-aware time series representation is needed to better utilize the LLM’s context window for TSAD. Instead of feeding raw data or verbose statistical blocks, developing token-efficient textual or embedding-based representations of temporal patterns will significantly enhance an agent’s reasoning capacity.
- **Agentic Parameter Selection:** Beyond merely selecting which algorithm to run, future agents could dynamically tune the specific hyperparameters of each detector (e.g., contamination rate, tree depth, or nearest neighbors) based on the evolving profile of the data.
- **Reinforcement Learning (RL):** The agentic workflow could be optimized using RL, where the agent receives

rewards based on threshold-independent metrics (like VUS-ROC). This would allow the agent to iteratively learn optimal tool-calling sequences and detector weighting strategies.

- **Time-Series Foundation Models:** As specialized time-series foundation models become more prevalent, integrating them as primary backend tools or fine-tuning them to natively support agentic tool-use could drastically reduce the reliance on traditional base detectors.

REFERENCES

- [1] LangChain AI. Langchain documentation, 2026.
- [2] Litsidis D. Atmatzidis G. TSAD-Orchestra: Agentic Model Selection for Time-Series Anomaly Detection. <https://github.com/dlitsidis/tsad-orchestra>, 2026.
- [3] Litsidis D. Atmatzidis G. TSAD-Orchestra: Agentic Model Selection for Time-Series Anomaly Detection. <https://tsad-orchestra.streamlit.app/>, 2026.
- [4] Paul Boniol, Ashwin K. Krishna, Marine Bruel, Qinghua Liu, Mingyi Huang, Themis Palpanas, Ruey S. Tsay, Aaron Elmore, Michael J. Franklin, and John Paparrizos. Vus: Effective and efficient accuracy measures for time-series anomaly detection. *arXiv preprint arXiv:2502.13318*, 2025.
- [5] Qinghua Liu, Paul Boniol, Themis Palpanas, and John Paparrizos. Time-series anomaly detection: Overview and new trends. *Proc. VLDB Endow.*, 17(12):4229–4232, August 2024.
- [6] Qinghua Liu, Seunghak Lee, and John Paparrizos. Tsb-autoad: Towards automated solutions for time-series anomaly detection. *Proceedings of the VLDB Endowment (PVLDB)*, 18(11):4364–4379, 2025.
- [7] Qinghua Liu and John Paparrizos. The elephant in the room: Towards a reliable time-series anomaly detection benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 108231–108261. Curran Associates, Inc., 2024.
- [8] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Raj Sinha, Jinwoo Shin, and Tomas Pfister. Ds-star: Data science agent for solving diverse tasks across heterogeneous formats and open-ended queries, 2026.
- [9] John Paparrizos and Michael J Franklin. Grail: efficient time-series representation learning. *Proceedings of the VLDB Endowment*, 12(11):1762–1777, 2019.
- [10] John Paparrizos and Luis Gravano. k-shape: Efficient and accurate clustering of time series. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pages 1855–1870, 2015.
- [11] Emmanouil Sylligardos, John Paparrizos, Themis Palpanas, Pierre Senellart, and Paul Boniol. Msad: A deep dive into model selection for time series anomaly detection, 10 2025.
- [12] Xiaoyu Tao, Yuchong Wu, Mingyue Cheng, Ze Guo, and Tian Gao. Anomamind: Agentic time series anomaly detection with tool-augmented reasoning. *arXiv preprint arXiv:2602.13807*, 2026.
- [13] Qitong Wang and Themis Palpanas. Deep learning embeddings for data series similarity search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, KDD '21*, page 1708–1716, New York, NY, USA, 2021. Association for Computing Machinery.
- [14] Phillip Wenig, Sebastian Schmidl, and Thorsten Papenbrock. Timeeval: a benchmarking toolkit for time series anomaly detection algorithms. *Proc. VLDB Endow.*, 15(12):3678–3681, August 2022.

A APPENDIX

A.1 Primary Agent Prompts

```
AGENT_SYSTEM_PROMPT = """\nYou are a time-series anomaly detection\norchestration agent operating in three phases.
```

PHASE 1: SCREENING

Profile the time series, then run 2-3 detector tools.

Each detector returns a compact STAT SUMMARY containing:

- . anomaly_candidates: how many points score above 0.5, 0.7, 0.9
- . score_percentiles: distribution (p50, p90, p95, p99)
- . hot_segments: index ranges [start, end] where scores peak above 0.5,
with the max score and count of points above 0.7

After reviewing all stat summaries, ASSESS each detector's signal quality:

- . INFORMATIVE: top_score > 0.5 AND has at least one hot_segment.
Include in ensemble.
- . UNINFORMATIVE: top_score <= 0.5 OR no hot_segments found.
Exclude from ensemble.

Form a hypothesis about WHERE anomalies are and which detector signals are strongest. You do NOT have access to the raw score values.

PHASE 2: DRILL-DOWN

Call drill_down_range(name, start, end, detectors) for each suspicious segment.

Use only the INFORMATIVE detectors identified in Phase 1.

It returns:

- . per_detector: top anomaly points (index, value, score) within the range
- . consensus_points: indices where ALL requested detectors score > 0.5 simultaneously
(the strongest possible signal; treat these as confirmed anomalies)

Inspect 2-4 hot segments, prioritising those with high max_score and multiple detectors agreeing.

PHASE 3: STORE ENSEMBLE

Call store_ensemble_scores(name, detectors) with ONLY the informative detectors you selected in Phase 1. Do NOT include detectors with flat or uninformative score distributions. This fuses cached scores server-side and persists the result. Do NOT skip this step.

FINAL REPORT

- . detectors_used: the short names matching what you passed to store_ensemble_scores.
- . summary: explain your screening observations, which detectors you excluded and why, which segments you drilled into, the ensemble consensus you found, and why you accepted or rejected borderline points.
Note: Confirmed anomaly points will be automatically extracted from your drill-down tool calls by the system.

Your output will be validated by a separate agent that checks it against the raw detector outputs, so your summary must accurately reflect the drill-down results.

```
AGENT_USER_PROMPT = """\nAnalyse time series '{series_id}' for anomalies\nusing the three-phase workflow:
```

1. profile_time_series('{series_id}'): understand scale, length, noise level.
2. Run 2-3 detector tools: review their stat summaries.
Identify which detectors are INFORMATIVE and which are UNINFORMATIVE.
3. drill_down_range('{series_id}', start, end, detectors): inspect the most suspicious segments using only your informative detectors.
4. store_ensemble_scores('{series_id}', detectors) : REQUIRED final tool call.
Pass ONLY the informative detectors.
5. Produce your final report: detectors_used and summary.

A.2 Validator Agent Prompts

```
VALIDATOR_SYSTEM_PROMPT = """\nYou are reviewing an anomaly detection report\nproduced by a three-phase screening,\ndrill-down, and store agent. Return a\nValidationResult.
```

Accept the report if:

- . At least 2 detectors were run.
- . At least one drill_down_range call was made on a suspicious segment.
- . store_ensemble_scores was called before finalizing.
- . Every anomaly in the report can be traced to a drill-down result.
- . The chosen detectors are a reasonable fit for the data characteristics.

Reject if:

- . Fewer than 2 detectors were run.
- . No drill-down was performed.
- . store_ensemble_scores was NOT called.
- . Anomalies are hallucinated.

```
. The detector selection clearly contradicts the  
data profile.
```

```
When rejecting, state exactly what is wrong and  
set severity to  
"minor", "major", or "critical".  
"""
```

```
VALIDATOR_USER_PROMPT = """\n  
Series ID: {series_id}
```

```
Iteration: {iteration}
```

```
RAW DETECTOR OUTPUTS:  
{context}
```

```
AGENT FINAL REPORT:  
{report_json}
```

```
Return a ValidationResult.  
"""
```